TECHNICAL UNIVERSITY - SOFIA

FACULTY OF COMPUTER SYSTEMS AND TECHNOLOGIES

BACHELOR THESIS ON TOPIC OF

DESIGN AND IMPLEMENTATION OF A MULTI-AGENT TASK SCHEDULING AND EXECUTION SYSTEM

ANSOR TAGHOEV

FACULTY NUMBER 273222034

2026

Table of Contents

1. INTRODUCTION.....4

2. PROBLEM DESCRIPTION AND SCOPE.....5

3. ANALYSIS OF EXISTING APPROACHES.....7

3.1. Centralized Scheduling.....7

3.2. Broker-Based Task Queues.....8

3.3. Database-Backed Task Queue.....8

3.4. Multi-Agent System Approach.....9

3.5. Comparative Summary.....10

4. GOALS, REQUIREMENTS, AND CONSTRAINTS.....11

4.1. Aim of the Thesis.....11

4.2. Main Development Tasks.....12

4.3. Functional Requirements.....13

4.4. Non-Functional Requirements.....15

4.5. Constraints and Out-of-Scope Features.....16

5. TECHNOLOGY OVERVIEW.....16

5.1. Java, JADE, and Maven.....17

5.2. PostgreSQL and JDBC.....17

5.3. Python Task Modules.....18

5.4. SQLAlchemy and Database Access in Python.....19

5.5. FastAPI, Jinja Templates, and the Web Dashboard.....20

5.6. Scripts, Configuration, and Testing Tools.....20

6. MULTI-AGENT SYSTEM DESIGN.....22

6.1. Agents and Responsibilities.....22

6.2. Agent Communication.....23

6.3. Task Scheduling Strategy.....24

6.4. Task Lifecycle and State Management.....25
7. SYSTEM IMPLEMENTATION.....28
7.1. Project Structure and Technology Stack.....28
7.2. Database Schema and Repository Layer.....28
7.3. JADE Agent Implementation.....30
7.4. Task Execution Layer.....31
7.5. Web Dashboard.....33
7.6. Running and Testing the System.....34
8. DEMONSTRATION AND RESULTS.....34
8.1. Demonstration Setup.....35
8.2. Demonstration Task Set.....35
8.3. Execution Results.....36
8.4. Dashboard Observation.....37
8.5. Test Results.....38
9. DISCUSSION.....39
9.1. Strengths of the Implemented System.....39
9.2. Limitations.....40
9.3. Possible Future Improvements.....41
10. CONCLUSION.....42
SOURCES.....44
APPENDIX.....45

1. INTRODUCTION

Modern software systems often consist of multiple interacting components that must coordinate their activities while operating with limited direct supervision. The agent-oriented approach models such components as autonomous software entities. Wooldridge and Jennings identify autonomy, social ability, reactivity, and proactiveness as central characteristics of intelligent agents [1]. When multiple agents interact within a shared environment, their coordination can be organized as a multi-agent system.

Multi-agent systems provide an approach for designing complex applications as organizations of interacting autonomous agents. This introduces abstractions such as agents, roles, responsibilities, interactions, and organizational structures [2]. By assigning separate responsibilities to different agents, a system can distribute control and allow individual components to react independently to changes in their environment.

Task scheduling and load balancing are important concerns in distributed systems. Task scheduling determines when and where work should be executed, while load balancing attempts to distribute work among the available processing resources. Poor scheduling decisions can leave some resources overloaded while others remain underused [3]. Scheduling strategies must therefore consider both the order in which tasks are processed and the selection of an appropriate execution resource.

This thesis presents the design and implementation of a multi-agent task scheduling and execution system. The proposed system separates scheduling and execution responsibilities between two agent roles. A coordinator agent retrieves pending tasks, prioritizes them, and assigns them to executor agents. Executor agents receive assignments and perform the requested work. Tasks and their lifecycle states are stored in PostgreSQL, which provides transactional operations for applying state changes as complete units [4].

The agent layer is implemented using the Java Agent DEvelopment Framework (JADE). JADE is an open-source middleware platform intended to simplify the development of interoperable multi-agent systems while complying with Foundation for Intelligent Physical Agents specifications [5]. It provides agent lifecycle management, behaviour abstractions, and communication through Agent Communication Language messages. These features make it suitable for demonstrating coordination and message exchange between the agents implemented in this project.

The principal objective of the thesis is to demonstrate how multi-agent concepts can be applied to task scheduling and execution. The developed system aims to:

- schedule pending tasks according to their priority;
- distribute tasks according to the current executor workload;
- support task execution through both Java and Python modules;
- maintain a persistent and traceable task lifecycle;
- handle unsuccessful execution through finite retries;
- recover tasks left in stale active states;
- demonstrate communication between cooperating JADE agents.

The remainder of the thesis describes the addressed scheduling problem, the design of the multi-agent architecture, its implementation with JADE, and the resulting system behaviour. It concludes with a discussion of the system's strengths, limitations, and possible future improvements.

2. PROBLEM DESCRIPTION AND SCOPE

Distributed software systems often need to execute multiple tasks over a set of available processing resources. In such systems, scheduling is the decision-making process that determines the order in which tasks are executed and the resources on which they run. Bittencourt et al. describe scheduling as a resource-sharing decision process that determines the execution order of activities over available resources [6]. Casavant and Kuhl also identify distributed scheduling as part of the broader resource management problem in distributed computing systems [7].

The problem addressed in this thesis is the scheduling and execution of independent tasks by a group of cooperating software agents. Each task has a type, input data, priority, lifecycle status, and execution result. A task must be selected from the pending queue, assigned to an available executor, executed, and then marked as completed or failed. This creates two connected problems: choosing which task should be processed next, and choosing which executor should receive it.

The current implementation focuses on independent tasks. This means that one task does not depend on the output or completion of another task. Such workloads are close to

what distributed-systems literature often calls “bag-of-tasks” applications, where individual tasks can be executed independently and do not require a strict dependency order [8]. This assumption keeps the project scope clear and allows the thesis to focus on task prioritization, executor selection, message exchange, lifecycle persistence, and recovery.

A simple centralized execution model would place both scheduling and execution responsibility in one component. While this is easier to implement, it limits the demonstration of cooperation between autonomous components and makes the system less representative of a multi-agent architecture. In contrast, the proposed system separates responsibilities between a coordinator agent and executor agents. The coordinator is responsible for selecting and assigning work, while executor agents are responsible for carrying out assigned tasks.

The scheduling strategy must also avoid inefficient workload distribution. If tasks are repeatedly assigned to the same executor while another executor remains idle, the system does not use its available resources effectively. For this reason, the project uses a least-loaded selection strategy: the coordinator checks how many active tasks are currently assigned to each executor and chooses the executor with the lowest active load. This strategy is intentionally simple, but it is suitable for demonstrating basic load-aware scheduling.

Another important part of the problem is reliable task state management. A task may be pending, assigned, running, completed, failed, or cancelled. These states must be stored persistently so that the system can be inspected and recovered after interruptions. PostgreSQL transactions are relevant here because they group database operations into all-or-nothing units and prevent incomplete intermediate states from becoming visible as final results [4].

The project must therefore solve the following practical requirements:

- store tasks persistently with their type, input data, priority, status, assigned executor, result, error, and timestamps;
- select pending tasks according to priority;
- assign tasks to executor agents according to current executor load;
- send task assignments through JADE agent messages;
- execute supported Java and Python task types;
- record successful completion and execution errors;
- retry failed tasks only while attempt limits allow it;

- recover stale assigned or running tasks after interrupted execution.

The scope of the thesis is limited to a local demonstration of a multi-agent scheduler. The system does not attempt to solve every form of distributed scheduling. It does not currently support task dependencies, deadlines, executor capability matching, distributed JADE containers on multiple physical machines, or dynamic executor discovery through the JADE Directory Facilitator. These features are left as possible future improvements. The goal of the project is to demonstrate a complete and understandable multi-agent scheduling system with persistent state, message-based coordination, and traceable task execution.

3. ANALYSIS OF EXISTING APPROACHES

This chapter compares several approaches that can be used to schedule and execute independent tasks. The comparison is used to justify the selected architecture of the implemented system.

3.1. Centralized Scheduling

One possible approach to the task execution problem is centralized scheduling. In this model, a single scheduling component observes the available tasks, chooses the next task to execute, and assigns it to an execution resource. Casavant and Kuhl describe scheduling as part of the broader resource management problem in distributed computing systems [7]. This makes centralized scheduling a natural baseline because all decisions are made by one component with a clear view of the task queue and the available executors.

Centralized scheduling is simple to understand and implement. The scheduler can apply one consistent policy, such as selecting the highest-priority pending task and assigning it to the least-loaded executor. This is also suitable for a local prototype, because it keeps the scheduling logic visible and easy to demonstrate. The system implemented in this thesis uses this idea through the coordinator agent.

The limitation of this approach is that the scheduler becomes a central point of control. If the scheduler is stopped, pending tasks cannot be assigned until it is started again. Executors may still finish tasks that were already assigned, but new scheduling decisions depend on the coordinator. This limitation is acceptable in the current project because the goal is to demonstrate a clear multi-agent scheduling workflow rather than a fully fault-tolerant distributed scheduler.

The workload considered in this thesis consists of independent tasks. Beaumont et al. discuss bag-of-tasks applications, where tasks are independent and can be executed without dependencies between them [8]. This matches the implemented system because each task can be selected, assigned, executed, and completed without waiting for another task's output. For this reason, a simple centralized scheduling strategy is appropriate for the scope of the project.

3.2. Broker-Based Task Queues

Another common approach is to use a broker-based task queue. In this model, clients place tasks into a queue, and worker processes consume tasks from that queue. Celery describes task queues as a mechanism for distributing work across threads or machines, with workers monitoring queues for work and messages usually passing through a broker [15]. This model is widely used for asynchronous background processing.

A broker-based queue has several advantages. It separates task submission from task execution, allows multiple workers to consume tasks, and often includes practical features such as retries, monitoring, scheduling, result storage, and worker management [15]. Frameworks such as Celery are mature and useful when the main goal is to execute background jobs reliably.

However, this thesis has a different focus. The goal is not only to process jobs, but to demonstrate a multi-agent system with explicit agent roles and message-based cooperation. If a ready-made task queue framework handled the assignment and delivery of tasks, much of the agent communication and scheduling logic would be hidden inside the framework. That would make the implementation less suitable for demonstrating how a coordinator agent and executor agents cooperate.

For this reason, a broker-based task queue is treated as an alternative approach rather than the selected architecture. It would be practical for a production background-job system, but it is less aligned with the educational and architectural goal of this thesis.

3.3. Database-Backed Task Queue

A third approach is to store tasks directly in a relational database and use task status fields to represent the execution lifecycle. In this model, tasks are represented as database rows. A scheduler or worker reads pending tasks, updates their status, assigns them to an executor, and later records completion or failure.

This approach fits the implemented system because PostgreSQL can store the complete task lifecycle. The database contains task type, input data, priority, status, assigned executor, attempt counters, timestamps, result data, and error information. PostgreSQL transactions are useful because related changes can be applied as complete units [4]. This helps avoid incomplete state changes, such as a task being partially assigned or partially completed.

Database-backed queues also make the system easier to inspect. A user can query the database or use the web dashboard to see which tasks are pending, assigned, running, completed, failed, or cancelled. The implemented system also records lifecycle events, which makes it possible to inspect not only the current task status, but also the sequence of status changes.

PostgreSQL also supports row-level locking. According to the PostgreSQL documentation, row-level locks block writers and lockers on the same row while still allowing normal data reading [16]. This is relevant for task queue designs because state transitions should protect individual task rows from conflicting updates. In the implemented system, conditional updates are used to ensure that a task changes state only when it is currently in the expected state.

The main limitation of a database-backed queue is that the database becomes a central shared dependency. For small and medium systems this is acceptable, especially when persistence and traceability are important. For very high-throughput distributed systems, a specialized broker may be more appropriate. In this thesis, the database-backed approach is appropriate because the project values persistent lifecycle state, simple inspection, and clear recovery behaviour.

3.4. Multi-Agent System Approach

The selected approach combines a database-backed task queue with a multi-agent architecture. Multi-agent systems organize software around interacting autonomous agents. Wooldridge and Jennings identify autonomy, social ability, reactivity, and proactiveness as important characteristics of agents [1]. Zambonelli, Jennings, and Wooldridge also describe agent-oriented design in terms of roles, responsibilities, interactions, and organizational structures [2].

In the implemented system, the coordinator and executor agents have separate responsibilities. The coordinator observes the task queue and assigns work. Executor agents receive assignments and perform the task execution. This separation makes the system more representative of a multi-agent design than a single monolithic scheduler-executor component.

JADE is used as the agent platform. Bellifemine et al. describe JADE as a framework for developing multi-agent applications and supporting agent communication according to FIPA specifications [5]. In this thesis, JADE provides the runtime environment for the coordinator and executor agents, while ACL messages are used to communicate task assignments.

The multi-agent approach is useful because it makes cooperation explicit. The coordinator does not directly execute task logic. Instead, it sends an assignment message to an executor. The executor then verifies the assignment through the database, starts the task, executes the corresponding logic, and records the final result. This creates a clear division between scheduling, communication, execution, and persistence.

The limitation of the current multi-agent implementation is that it remains intentionally small. Executor agents are configured manually, and dynamic discovery through the JADE Directory Facilitator is not implemented. The system also uses one coordinator, so scheduling is not distributed among multiple coordinator agents. These limitations keep the project understandable and demonstrable, while leaving space for future improvements.

3.5. Comparative Summary

The comparison shows that each approach has a different role. A centralized scheduler is simple and suitable for clear decision-making. A broker-based task queue is practical for production background processing, but it hides much of the scheduling and communication logic. A database-backed queue provides persistence and traceability. A multi-agent system makes the cooperation between software entities explicit.

The implemented system combines the centralized scheduler, database-backed queue, and multi-agent approaches. The coordinator agent performs centralized scheduling. PostgreSQL stores the persistent task lifecycle. JADE provides the agent runtime and message exchange. This combination supports the main goal of the thesis: to demonstrate a working multi-agent task scheduling and execution system with persistent state, visible lifecycle transitions, and cooperating agents.

Approach	Advantages	Limitations	Relevance to this thesis
Centralized scheduler	Simple control logic; clear scheduling decisions	Depends on one scheduling component	Used through the coordinator agent
Broker-based task queue	Mature worker model; supports asynchronous execution	Agent communication is mostly hidden by the framework	Discussed as an alternative
Database-backed queue	Persistent task state; easy inspection; supports recovery	Database is a central dependency	Used as the source of truth
Multi-agent system	Clear agent roles; explicit communication; extensible design	More complex than a simple worker queue	Main architectural approach

Table 1: Comparison of task execution approaches.

4. GOALS, REQUIREMENTS, AND CONSTRAINTS

4.1. Aim of the Thesis

The main aim of this thesis is to design and implement a multi-agent task scheduling and execution system. The system demonstrates how agent-oriented concepts can be applied to a practical scheduling problem in which independent tasks must be selected, assigned, executed, and tracked. The project is based on two main agent roles: a coordinator agent and executor agents.

The coordinator agent is responsible for scheduling decisions. It observes the task queue, selects eligible pending tasks, chooses an executor according to the current executor load, and sends task assignment messages. Executor agents are responsible for receiving assignments, starting the assigned task, executing the requested task logic, and recording the result. This separation follows the general multi-agent idea that software systems can be

organized through interacting autonomous entities with distinct roles and responsibilities [1], [2].

The goal of the project is not to build a production-grade distributed task platform. Instead, the goal is to create a complete and understandable prototype that demonstrates scheduling, message exchange, persistent task state, retries, stale-task recovery, and dashboard-based observation. The implementation is therefore intentionally focused on clarity and traceability.

4.2. Main Development Tasks

To achieve the aim of the thesis, the project is divided into several development tasks. These tasks correspond to the main parts of the implemented system: task storage, scheduling, agent communication, task execution, monitoring, and testing.

The first development task is to design a task model that can represent the lifecycle of a task. Each task must contain enough information to support scheduling and execution. This includes the task type, input data, priority, current status, assigned executor, attempt counters, result data, error data, and timestamps.

The second development task is to implement persistent task storage. PostgreSQL is used as the database layer because the task lifecycle must remain inspectable after the agents stop or restart. Transactions are important in this part of the system because task state changes must be applied consistently [4].

The third development task is to implement the coordinator agent. The coordinator must read pending tasks, apply task ordering, inspect executor load, assign a task in the database, and send an assignment message to the selected executor. This task connects the scheduling strategy with the agent communication layer.

The fourth development task is to implement executor agents. Each executor must listen for task assignment messages, verify the assigned task through the database, mark the task as running, execute the corresponding task logic, and store either a successful result or failure information.

The fifth development task is to support multiple execution mechanisms. The implemented system supports Java-native task types and Python-backed task types. This

shows that executor agents can coordinate task execution even when the actual task logic is implemented in different runtimes.

The sixth development task is to implement monitoring and demonstration tools. The web dashboard provides a visual way to inspect task status, executor workload, task details, and lifecycle events. This is important because the behaviour of a multi-agent system should be observable during demonstration.

The final development task is to test the main execution logic. Automated tests are used to verify Java task execution and Python task modules. These tests do not replace the full system demonstration, but they provide evidence that important execution components behave correctly.

4.3. Functional Requirements

The functional requirements describe what the system must be able to do. They are derived from the scheduling problem, the selected multi-agent architecture, and the implemented project scope.

Requirement	Description
FR1	The system must store tasks persistently in PostgreSQL.
FR2	The system must represent task lifecycle states such as PENDING, ASSIGNED, RUNNING, COMPLETED, FAILED, and CANCELLED.
FR3	The coordinator agent must select pending tasks according to priority and creation time.
FR4	The coordinator agent must choose an executor according to the current active executor load.
FR5	The coordinator agent must send task assignments through JADE ACL messages.

FR6	Executor agents must receive assignment messages and verify task ownership through the database.
FR7	Executor agents must execute supported Java-native task types.
FR8	Executor agents must execute supported Python-backed task types through the Python task runner.
FR9	The system must store successful task results.
FR10	The system must store execution errors when task execution fails.
FR11	The system must retry failed tasks only while remaining attempts are available.
FR12	The system must mark tasks as failed when the maximum number of attempts is reached.
FR13	The system must recover stale assigned or running tasks after interrupted execution.
FR14	The system must expose task status and lifecycle information through the web dashboard.
FR15	The system must provide scripts for running agents, creating tasks, and demonstrating the workflow.

Table 2: Functional requirements of the implemented system.

These requirements define the expected behaviour of the implemented system. The most important functional requirement is the correct task lifecycle, because scheduling, execution, retry handling, stale recovery, and dashboard observation all depend on reliable task state changes.

4.4. Non-Functional Requirements

In addition to functional requirements, the system must satisfy several non-functional requirements. These describe qualities of the implementation rather than individual features.

One non-functional requirement is traceability. The system should make task execution history visible. This is supported through persistent task status fields and lifecycle events. A user should be able to inspect whether a task was created, assigned, started, completed, retried, recovered, failed, or cancelled.

Another requirement is reliability within the local demonstration scope. The system should avoid invalid task transitions, duplicated starts, and permanently stuck active tasks. Conditional database updates help ensure that a task changes state only when it is in the expected previous state. Stale-task recovery also helps restore tasks left in active states after interrupted execution.

A third requirement is simplicity. The scheduling strategy should be understandable and demonstrable. For this reason, the project uses priority ordering and least-loaded executor selection instead of a more complex scheduling algorithm. This keeps the scheduling behaviour clear enough to explain and verify.

Maintainability is also important. The implementation separates responsibilities across repository classes, agent classes, behaviour classes, execution classes, scripts, and web dashboard modules. This separation makes it easier to understand which part of the system is responsible for scheduling, message handling, execution, database access, or visualization.

Demonstrability is another requirement. Since this is a thesis project, the system must be easy to present. The dashboard, demo task scripts, lifecycle events, and final task statuses all help demonstrate the behaviour of the system without requiring the reader to inspect the database manually.

4.5. Constraints and Out-of-Scope Features

The project has several constraints. The first constraint is that the system is implemented as a local demonstration. The coordinator agent, executor agents, database, and dashboard are intended to be run in a controlled local environment. The thesis does not evaluate deployment across multiple physical machines.

The second constraint is that tasks are independent. The system does not support workflows where one task depends on the result of another task. This keeps the scheduling problem focused on priority, executor load, task execution, retries, and recovery.

The third constraint is that executor agents are configured manually. The current implementation does not use the JADE Directory Facilitator for dynamic executor registration and discovery. Dynamic discovery is therefore treated as a future improvement rather than a core requirement.

The fourth constraint is that the scheduling strategy is intentionally simple. The coordinator does not estimate task duration, task complexity, deadlines, executor capabilities, or historical executor performance. All configured executors are treated as equivalent except for their current active task load.

The fifth constraint is that the dashboard is used for observation and task creation, not for executing tasks directly. Scheduling and execution remain the responsibility of the JADE agents. This preserves the separation between the monitoring interface and the multi-agent execution layer.

These constraints define the scope of the thesis. They prevent the project from becoming too broad while still allowing it to demonstrate the main concepts: multi-agent coordination, persistent task lifecycle management, message-based assignment, task execution, retry handling, stale recovery, and result inspection.

5. TECHNOLOGY OVERVIEW

This chapter introduces the main technologies used in the implemented system. The purpose of the chapter is not to describe every library in detail, but to explain why each technology is relevant to the project and how it supports the multi-agent scheduling workflow.

5.1. Java, JADE, and Maven

The agent layer of the system is implemented in Java. Java is used for the coordinator agent, executor agents, task repository access, and task execution coordination. This choice is appropriate because JADE is a Java-based multi-agent framework. Bellifemine et al. describe JADE as a software framework for developing multi-agent applications and supporting agent communication according to FIPA specifications [5].

JADE is the central technology for the agent part of the project. It provides the runtime environment in which the coordinator and executor agents are started. It also provides abstractions for agent behaviours and Agent Communication Language messages. In the implemented system, the coordinator uses a periodic scheduling behaviour, while executor agents use a receiver behaviour that waits for incoming task assignment messages.

The project does not use JADE only as a background library. JADE directly shapes the system architecture. The separation between coordinator and executor agents corresponds to the agent-oriented model discussed in earlier chapters. The coordinator is responsible for scheduling decisions, while executor agents are responsible for executing assigned tasks. Communication between these roles is performed through JADE ACL messages.

The Java part of the project is built with Apache Maven. Maven documentation provides guides for the build lifecycle, the Project Object Model, dependency management, repositories, and plugin configuration [18]. In this project, the Maven configuration defines the Java version, dependencies, and plugins required to compile, test, and run the agent system.

The Maven project includes dependencies for JADE, the PostgreSQL JDBC driver, Jackson, and JUnit. JADE supports the agent runtime and communication layer. The PostgreSQL JDBC driver allows Java code to connect to the database. Jackson is used for JSON processing inside the Java task execution layer. JUnit is used for automated testing of the Java execution logic.

5.2. PostgreSQL and JDBC

PostgreSQL is used as the persistent storage layer of the system. It stores task records, task lifecycle state, execution results, error messages, assigned agents, attempt

counters, timestamps, and lifecycle events. This makes PostgreSQL more than a simple storage component. It acts as the persistent source of truth for the multi-agent workflow.

The use of PostgreSQL is important because task state must survive agent restarts. If the coordinator or executor agents stop, the current task status remains stored in the database. When the coordinator starts again, stale assigned or running tasks can be recovered according to the lifecycle rules of the system.

PostgreSQL transactions are relevant because task state changes must be applied consistently. The PostgreSQL documentation describes transactions as a way to group operations into all-or-nothing units [4]. In the implemented system, this is important for operations such as assigning a pending task, marking a task as running, completing a task, or handling execution failure.

The database schema also uses PostgreSQL triggers to record lifecycle events. PostgreSQL documentation describes triggers as database operations that can automatically execute functions when table events occur [10]. In this project, a trigger records events such as task creation, assignment, execution start, completion, failure, retry scheduling, recovery, and cancellation.

Java communicates with PostgreSQL through the PostgreSQL JDBC driver. The PostgreSQL JDBC documentation describes the driver as the Java interface used to connect Java applications to PostgreSQL databases [11]. In the implemented system, JDBC is used by the Java repository layer to read pending tasks, assign tasks, update task state, count executor load, and store results or errors.

5.3. Python Task Modules

Python is used for part of the task execution layer. The system supports both Java-native and Python-backed task types. Java-native task types are executed directly inside the Java agent system. Python-backed task types are delegated to a separate Python script.

The Python task runner currently supports PYTHON_ECHO, TEXT_STATS, and KEYWORD_COUNT. PYTHON_ECHO returns a message from the input data. TEXT_STATS calculates basic text statistics, including character count, word count, and line count. KEYWORD_COUNT counts case-insensitive whole-word occurrences of a keyword in a text.

This design demonstrates that the executor agent can coordinate work implemented in another runtime. The executor agent remains responsible for the task lifecycle, while the Python script is responsible only for the task-specific logic. This keeps scheduling and state management inside the agent system while still allowing task logic to be extended in Python.

The Java implementation starts the Python task runner as a separate operating system process. The Java ProcessBuilder API is designed for creating operating system processes and controlling attributes such as command arguments, environment, working directory, and standard input/output streams [17]. In this project, the Python bridge uses this mechanism to start the Python runner, pass task input, wait for completion, and collect the output.

A timeout is used when running Python-backed tasks. This prevents a Python task from running indefinitely and blocking the executor agent. If the Python process finishes successfully, the output is stored as the task result. If it exits with an error or times out, the executor handles the failure through the same retry and failure lifecycle used by Java-native tasks.

5.4. SQLAlchemy and Database Access in Python

The project also includes Python database access modules. SQLAlchemy is used for the web dashboard and supporting scripts. SQLAlchemy documentation describes sessions as the main mechanism for ORM operations and transaction management in application code [12].

In the implemented system, SQLAlchemy models represent the tasks and task_events tables. The Task model contains fields such as task type, input data, priority, status, assigned agent, attempt counters, result, error, and timestamps. The TaskEvent model stores lifecycle event records linked to tasks.

Using SQLAlchemy in the Python layer provides a convenient way for the dashboard and scripts to interact with the same PostgreSQL schema used by the Java agents. This allows the system to expose task state through a web interface without duplicating the task model manually in every route or script.

The Python repository layer is mainly used for task creation and inspection from the dashboard and command-line scripts. Scheduling and execution remain the responsibility of the Java JADE agents. This separation is important because the dashboard should not become

a hidden scheduler. Its role is to create tasks and display state, while the agents perform the actual multi-agent workflow.

5.5. FastAPI, Jinja Templates, and the Web Dashboard

FastAPI is used to implement the web dashboard. FastAPI documentation describes it as a framework for building APIs with Python based on standard Python type hints [13]. In this project, FastAPI provides routes for the dashboard page, task queue page, task creation form, task detail page, and demo task creation.

The dashboard helps demonstrate the system. It shows task counts grouped by status, active executor workload, recent lifecycle events, recent tasks, filtered task lists, and detailed task information. This makes the internal behaviour of the multi-agent system visible during execution.

The dashboard uses Jinja templates for rendering HTML pages. FastAPI provides support for templates, and its documentation includes a section on using templates in FastAPI applications [14]. Jinja documentation explains that templates are text files containing variables or expressions that are replaced with values when the template is rendered [19].

In the implemented system, Jinja templates are used to separate presentation from route logic. The FastAPI routes prepare data such as task rows, counts, events, and selected filters. The templates then render this information into HTML pages. This keeps the dashboard easier to maintain because database queries and page layout are not mixed into one large file.

The dashboard is intentionally limited to observation and task creation. It does not replace the coordinator and executor agents. This design preserves the separation between the web interface and the multi-agent execution layer. A user can create a task from the dashboard, but the task is still scheduled and executed by the JADE agents.

5.6. Scripts, Configuration, and Testing Tools

The project includes shell and Python scripts for running and demonstrating the system. The run_agents.sh script starts the JADE platform with two executor agents and one coordinator agent. The run_web.sh script starts the FastAPI dashboard. Other scripts create

tasks, apply database migrations, list tasks, list lifecycle events, monitor task state, and run integration tests.

Configuration is handled through environment variables and an optional .env file. This allows database connection settings, executor names, coordinator name, timeout values, and other runtime values to be changed without modifying the source code. This is useful for a local demonstration because the same code can be run with different database or agent settings.

Testing is implemented in both Java and Python. Java tests verify task execution behaviour in the Java task executor. Python tests verify the Python task modules. The project also contains opt-in database integration tests, which are separated from the default test run because they require a configured PostgreSQL environment.

These tools support the demonstrability and maintainability of the project. The scripts make it easier to start the required components in a repeatable way. The tests provide evidence that individual execution components behave correctly. Together, they make the project more reliable as a thesis prototype.

Technology	Role in the system
Java	Implements JADE agents, scheduling logic, repository access, and task execution coordination
JADE	Provides the multi-agent runtime, behaviours, and ACL message exchange
Maven	Builds the Java agent system and manages Java dependencies
PostgreSQL	Stores tasks, lifecycle state, results, errors, and lifecycle events
JDBC	Connects the Java agent system to PostgreSQL
Python	Implements selected task modules and support scripts
SQLAlchemy	Provides Python ORM access to the PostgreSQL task schema
FastAPI	Implements the web dashboard routes
Jinja	Renders dashboard HTML templates

Shell sctipts	Start agents, start the dashboard, and support demonstrations

Table 3: Main technologies used in the implemented system.

6. MULTI-AGENT SYSTEM DESIGN

The proposed system is designed as a small multi-agent architecture in which task scheduling and task execution are separated into different agent roles. This separation follows the agent-oriented idea of organizing a system around agents, roles, responsibilities, and interactions [2]. In the context of this thesis, the agents do not attempt to model complex reasoning or negotiation. Instead, they cooperate through clearly defined responsibilities and a simple message-based protocol.

The system contains two main agent roles: a coordinator agent and executor agents. The coordinator is responsible for observing pending tasks, selecting the next task to process, choosing an executor, and sending the assignment. Executor agents are responsible for receiving assignments, starting the assigned task, executing the corresponding task logic, and recording the result. PostgreSQL is used as the persistent shared state between the agents, while JADE provides the agent runtime and communication mechanism [5].

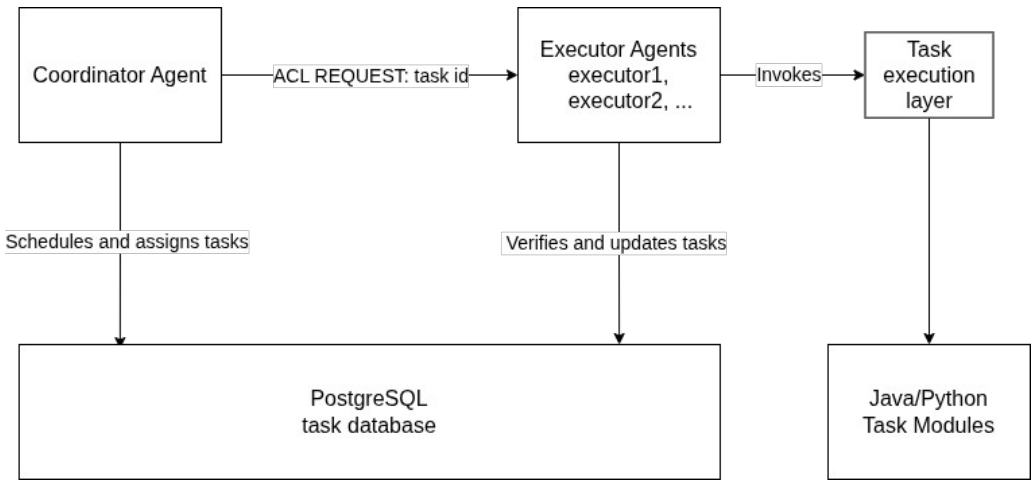

Figure 1: Architecture Diagram

6.1. Agents and Responsibilities

The coordinator agent represents the scheduling part of the system. Its responsibility is to periodically inspect the task queue and assign eligible work to an executor. On startup, it

first performs stale-task recovery so that tasks left in active states from a previous interrupted run can be returned to a valid state. After that, it performs an immediate scheduling attempt and then continues scheduling at a fixed interval.

The executor agent represents the execution part of the system. Each executor waits for task assignment messages. When an assignment is received, the executor verifies through the database that the task is assigned to its own agent name. If the task can be started, it is marked as running and passed to the task execution layer. After execution, the executor records either a successful result or a failure state.

The database is not an agent, but it is a central part of the design. It stores task data, lifecycle status, assigned executor, attempt counters, results, errors, and timestamps. This makes the state of the system persistent and inspectable. It also prevents the message protocol from carrying all task data, because the message only needs to contain the task identifier.

Component	Responsibility
Coordinator agent	Finds pending tasks, selects executor, assigns work
Executor agent	Receives assignment, runs task, records result
PostgreSQL database	Stores task state and lifecycle history
Task execution layer	Executes Java or Python task logic

Table 4: System components and responsibilities.

6.2. Agent Communication

Communication between agents is performed through JADE ACL messages. JADE provides support for the FIPA Agent Communication Language through the jade.lang.acl package, including the ACLMessage class [9]. In the implemented protocol, the coordinator sends an ACL message with the REQUEST performative to the selected executor. The message content contains the task identifier.

The message uses the conversation id task-assignment. This field separates task assignment messages from any other possible messages that an executor might receive in the future. If an executor receives a message with a different conversation id, it ignores the message. This keeps the current protocol simple while leaving room for future extensions.

The database remains the source of truth for task ownership. The coordinator first assigns the task in PostgreSQL and only then sends the JADE message. The executor does not trust the message alone; it attempts to mark the task as running only if the database confirms that the task is assigned to that executor. This protects the lifecycle from incorrect or duplicated execution.

6.3. Task Scheduling Strategy

The scheduling strategy has two decisions: which task should be processed next, and which executor should receive it. The coordinator reads pending tasks from PostgreSQL and relies on repository ordering to prioritize them. Pending tasks are ordered by priority in descending order and then by creation time in ascending order. This means that higher-priority tasks are preferred, while tasks with the same priority are processed in the order in which they were created.

After selecting the next pending task, the coordinator chooses an executor using a least-loaded strategy. For each configured executor name, it counts active tasks in the ASSIGNED and RUNNING states. The executor with the lowest active count is selected. This is a simple load-aware strategy: it does not estimate task duration or executor performance, but it prevents the coordinator from assigning all work to the same executor while another configured executor is idle.

The assignment itself is performed through a conditional database update. A task can be assigned only if it is still in the PENDING state and has remaining attempts. If the task has already been changed by another process, the assignment returns no task and no message is sent. This design keeps the scheduling decision simple while still protecting the task lifecycle against duplicate assignment.

The strategy can be summarized as follows:

1. Read up to 10 pending tasks.
2. If no pending task exists, do nothing.
3. Select the first task from priority-ordered results.
4. Count active tasks for each configured executor.
5. Select the executor with the lowest active count.
6. Atomically assign the task in PostgreSQL.

7. Send a JADE REQUEST message containing the task id.

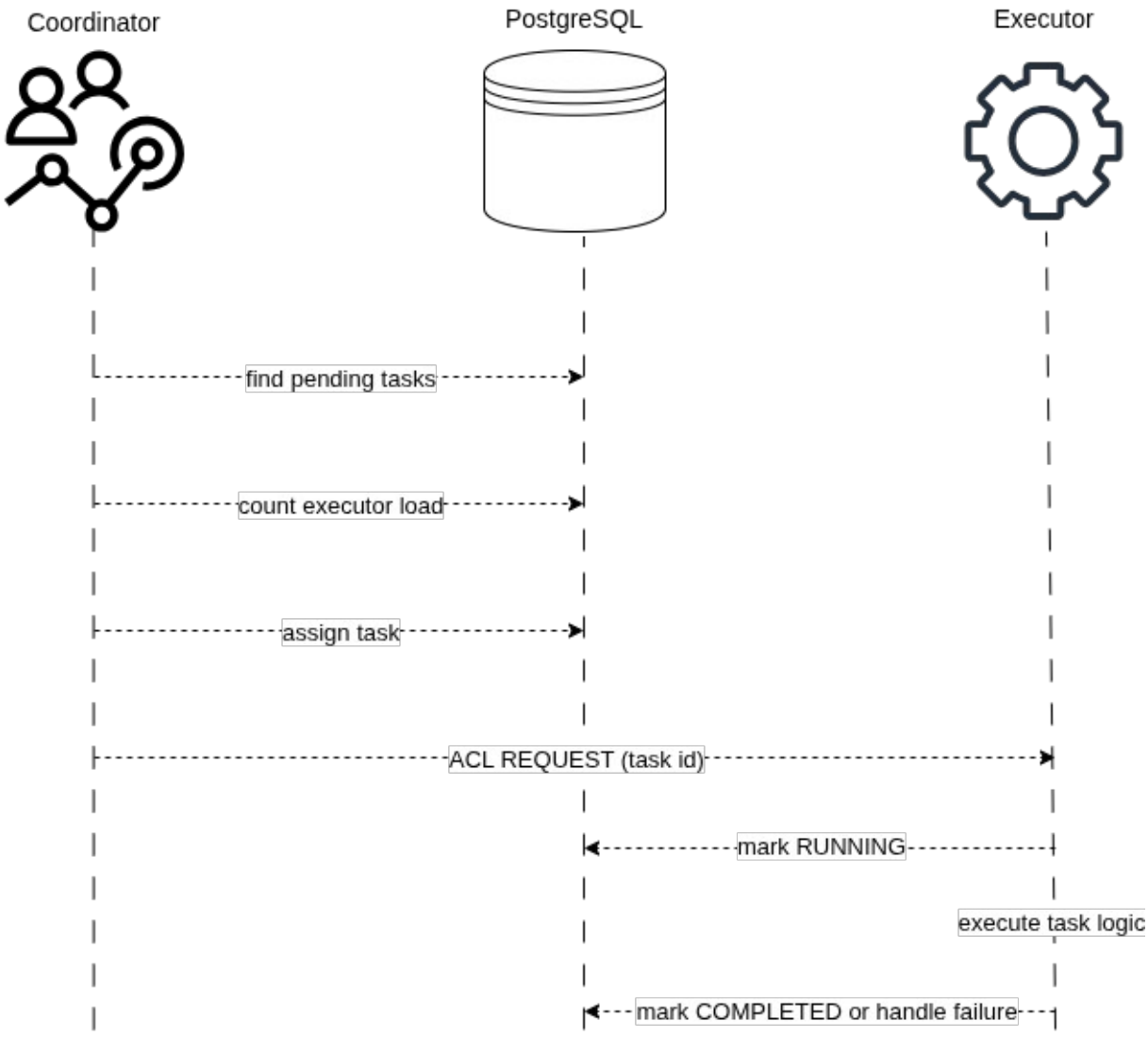

Figure 2: Task assignment and execution sequence.

This scheduling strategy is appropriate for the scope of the thesis because it is understandable, deterministic, and directly demonstrable. More advanced strategies, such as task-duration estimation, deadline-aware scheduling, executor capability matching, or dynamic discovery through the JADE Directory Facilitator, are left for future work.

6.4. Task Lifecycle and State Management

The task lifecycle defines the valid states through which a task can move from its creation to its final result. PostgreSQL acts as the source of truth for this lifecycle. Each task

record contains its current status, assigned executor, attempt counters, result or error information, and timestamps corresponding to important lifecycle transitions.

A newly created task begins in the PENDING state. In this state, it is eligible for scheduling while its attempt_count remains lower than its configured max_attempts. When the coordinator selects the task and successfully assigns it to an executor, the task moves to ASSIGNED. The assigned executor then verifies its ownership through the database before changing the task to RUNNING. The attempt counter is incremented only when execution begins.

The normal successful lifecycle is:

PENDING -> ASSIGNED -> RUNNING -> COMPLETED

When execution completes successfully, the executor stores the result, changes the status to COMPLETED, and records the completion time. COMPLETED is a terminal state, meaning that the task is no longer eligible for scheduling or execution.

Execution failures follow a finite retry policy. If an execution attempt fails while additional attempts remain, the task returns from RUNNING to PENDING. Its assignment and start timestamps are cleared, allowing the coordinator to schedule another attempt. If the task has reached its maximum number of attempts, it instead moves to the terminal FAILED state and stores the corresponding error message.

RUNNING -- failure, attempts remain --> PENDING

RUNNING -- failure or stale, no attempts remain --> FAILED

The system also supports recovery of stale tasks. A task is considered stale when it remains in the ASSIGNED or RUNNING state beyond the configured recovery threshold. During coordinator startup, stale tasks with remaining attempts are returned to PENDING. Tasks that have exhausted their available attempts are changed to FAILED. This prevents interrupted assignments or executions from remaining permanently active.

The CANCELLED status is another terminal state. It is intended for tasks that are deliberately archived or excluded from execution. Unlike COMPLETED and FAILED, cancellation is currently performed through maintenance tools rather than through the normal agent workflow.

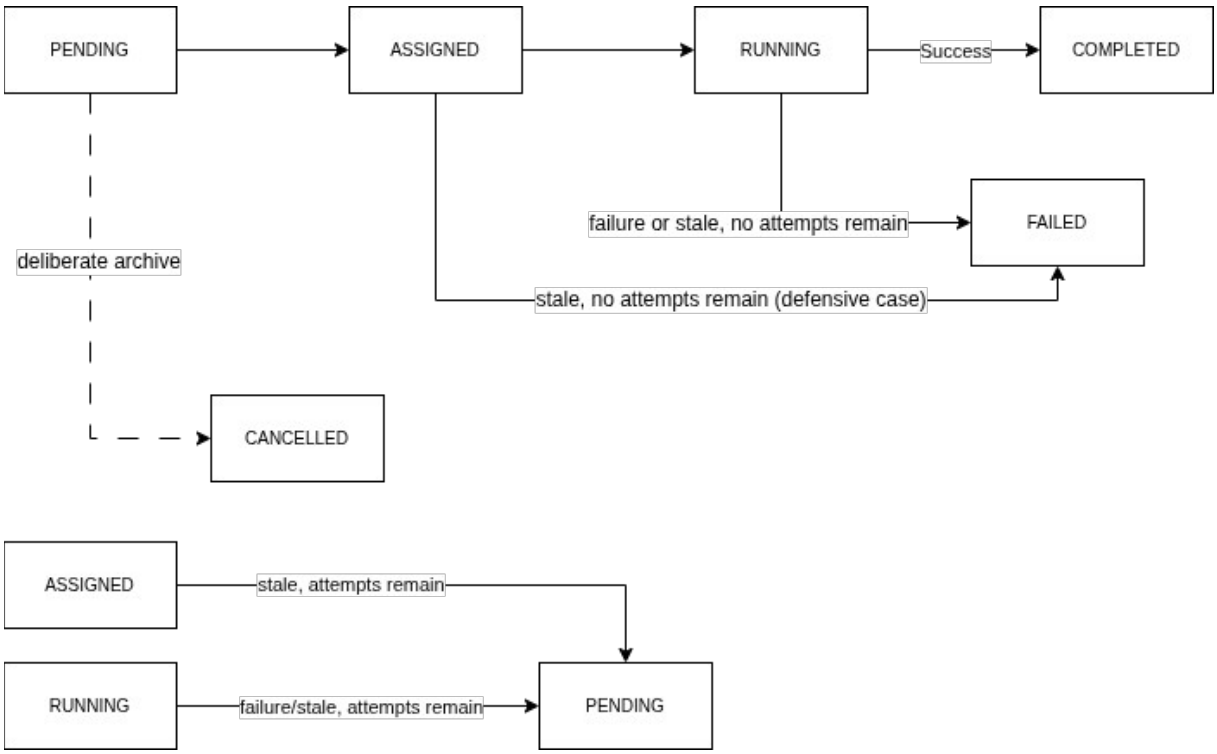

Figure 3: Task Lifecycle Diagram

Note: Repeated state boxes represent the same lifecycle states and are shown only to keep the transition arrows readable.

Task state transitions are implemented as conditional PostgreSQL updates. Each operation verifies the expected current status and, where necessary, the assigned executor. If those conditions no longer match, the update does not modify the task. This prevents an executor from starting a task assigned to another agent and reduces the risk of invalid or duplicated transitions. PostgreSQL transactions ensure that related database operations are treated as complete units [4].

The system additionally maintains a structured lifecycle history in the task_events table. A PostgreSQL trigger records task creation and each status transition, including assignment, execution start, completion, failure, retry scheduling, recovery, and cancellation. PostgreSQL triggers execute as part of the same transaction as the statement that activated them, meaning that the task update and its corresponding event are committed or rolled back together [10]. This event history makes task execution traceable and supports monitoring and later analysis.

7. SYSTEM IMPLEMENTATION

This chapter describes the implementation of the multi-agent task scheduling system. The implementation is divided into several cooperating layers: a PostgreSQL database layer, a Java-based JADE agent layer, a task execution layer, Python task modules, command-line support scripts, and a web dashboard. The database stores task state and lifecycle history. The JADE agents perform scheduling and execution. The dashboard provides a visual interface for observing the queue, creating tasks, and inspecting task results.

7.1. Project Structure and Technology Stack

The implementation is organized into separate project directories according to responsibility. The agent-system directory contains the Java 17 and Maven-based JADE agent implementation. The db directory contains Python SQLAlchemy models and repository helpers for accessing the PostgreSQL task schema. The python_tasks directory contains Python-backed task execution logic. The scripts directory contains migration, execution, monitoring, and maintenance commands. The web directory contains the FastAPI dashboard, Jinja templates, and static styling.

The Java agent layer communicates with PostgreSQL through the PostgreSQL JDBC driver. The pgJDBC documentation describes the driver as allowing Java programs to connect to PostgreSQL using standard database-independent Java code [11]. The Python parts of the system use SQLAlchemy sessions for database interaction; SQLAlchemy describes the Session as the object that establishes conversations with the database and manages ORM-mapped objects during its lifespan [12]. The web dashboard is implemented with FastAPI, a Python web framework based on standard Python type hints [13], and uses FastAPI support for Jinja templates and static files [14].

7.2. Database Schema and Repository Layer

The central database table is public.tasks. Each task record contains the task type, JSON input data, priority, current status, assigned agent, attempt counters, execution result, error message, and lifecycle timestamps. The schema is created and upgraded by the migration script scripts/migrate_tasks_multi_agent.py. The migration is idempotent, meaning it can be run repeatedly without recreating existing objects unnecessarily.

The database also contains a task_events table. This table stores a history of lifecycle events for each task, such as task creation, assignment, execution start, completion, retry scheduling, recovery, failure, and cancellation. A PostgreSQL trigger records these events whenever a task is inserted or its status changes. This design keeps lifecycle history close to the task state itself and uses PostgreSQL transaction behaviour so that task updates and their corresponding events are committed together [4], [10].

The Java-side database access is implemented in TaskRepository. This class performs conditional SQL updates for assignment, execution start, completion, failure handling, and stale-task recovery. For example, a task can be assigned only if it is still PENDING and has remaining attempts. An executor can mark a task as RUNNING only if the task is assigned to that executor. These conditions protect the system from invalid transitions and reduce the risk of duplicated execution.

```
String sql = """
    update public.tasks
    set status = ?,
        assigned_agent = ?,
        assigned_at = now(),
        result = null,
        error = null,
        finished_at = null
    where id = ? and status = ? and attempt_count < max_attempts
    returning
        id, task_type, input_data::text as input_data_json,
        priority, attempt_count, max_attempts, status,
        assigned_agent, assigned_at, result, error,
        created_at, started_at, finished_at
    """;
```

Listing 1: Conditional task assignment query.

The repository layer uses conditional SQL updates to protect task state transitions. Listing 1 shows the assignment operation. A task is assigned only if it is still in the PENDING state and still has remaining attempts. If another process has already changed the task, the update returns no row and the coordinator does not send an assignment message.

7.3. JADE Agent Implementation

The agent layer contains two main agent classes: CoordinatorAgent and ExecutorAgent. The CoordinatorAgent is responsible for scheduling. On startup, it performs stale-task recovery, attempts one immediate scheduling operation, and then registers a periodic scheduling behaviour. The scheduling interval is implemented through CoordinatorSchedulingBehaviour, which extends JADE TickerBehaviour and calls the coordinator's scheduling method repeatedly.

During each scheduling attempt, the coordinator reads pending tasks from PostgreSQL, selects the highest-priority pending task, counts active ASSIGNED and RUNNING tasks for each configured executor, and chooses the least-loaded executor. After the database assignment succeeds, the coordinator sends an ACL REQUEST message to the selected executor. The message uses the conversation id task-assignment and contains only the task id. JADE provides support for ACL message exchange through its jade.lang.acl package [9].

The ExecutorAgent is responsible for receiving and executing assignments. It registers TaskAssignmentReceiverBehaviour, which extends JADE CyclicBehaviour. This behaviour waits for incoming ACL messages, ignores messages with unrelated conversation ids, extracts the task id from valid assignment messages, and delegates execution to the executor agent. Before running task logic, the executor verifies ownership through the database by changing the task from ASSIGNED to RUNNING.

```
private void sendAssignmentMessage(Task task, String executorName) {
    ACLMessage message = new ACLMessage(ACLMessage.REQUEST);
    message.addReceiver(new AID(executorName, AID.ISLOCALNAME));
    message.setConversationId("task-assignment");
    message.setContent(Long.toString(task.getId()));
    send(message);
}
```

Listing 2: Sending a task assignment through a JADE ACL message.

After a task is assigned in PostgreSQL, the coordinator sends a JADE ACL message to the selected executor. Listing 2 shows that the message uses the REQUEST performative, contains the task identifier, and is marked with the task-assignment conversation id.

```
ACLMessage message = executorAgent.receive();
if (message == null) {
    block();
    return;
}

if (!TASK_ASSIGNMENT_CONVERSATION_ID.equals(message.getConversationId())) {
    return;
}

long taskId = Long.parseLong(message.getContent());
executorAgent.executeTask(taskId);
```

Listing 3: Receiving and filtering task assignment messages.

The executor agent receives messages through a cyclic behaviour. Listing 3 shows that messages with unrelated conversation ids are ignored. Valid task-assignment messages are parsed and passed to the executor agent for execution.

7.4. Task Execution Layer

Task execution is implemented through the TaskExecutor class. This class selects execution logic according to the task_type stored in the database. The Java-native task types are ECHO and TEXT_SUMMARY. ECHO returns the message field from the task input. TEXT_SUMMARY normalizes whitespace and truncates text according to the optional max_length value.

Python-backed tasks are executed through PythonTaskRunner. This class starts the python_tasks/run_task.py script as a separate process, passes the task type and input JSON as command-line arguments, waits for completion, and returns the script output as the task result. The supported Python-backed task types are PYTHON_ECHO, TEXT_STATS, and KEYWORD_COUNT. A timeout is used so that a Python task cannot run indefinitely.

If task execution succeeds, the executor stores the result and marks the task as COMPLETED. If execution fails, the executor calls the repository failure handler. Tasks with

remaining attempts are returned to PENDING, while tasks that have reached their attempt limit are marked FAILED. This keeps execution failure handling inside the same lifecycle rules described in Section 6.4.

```
public String execute(Task task) throws TaskExecutionException {
    if (ECHO_TASK_TYPE.equals(task.getTaskType())) {
        return executeEcho(task);
    }
    if (TEXT_SUMMARY_TASK_TYPE.equals(task.getTaskType())) {
        return executeTextSummary(task);
    }
    if (PYTHON_ECHO_TASK_TYPE.equals(task.getTaskType())
        || TEXT_STATS_TASK_TYPE.equals(task.getTaskType())
        || KEYWORD_COUNT_TASK_TYPE.equals(task.getTaskType())) {
        return pythonTaskRunner.run(task.getTaskType(), task.getInputDataJson());
    }

    throw new TaskExecutionException("Unsupported task type: " + task.getTaskType());
}
```

Listing 4: Task execution dispatch by task type.

Task execution is selected according to the task_type field stored in the database. Listing 4 shows the dispatch logic. Java-native task types are handled directly, while Python-backed task types are delegated to the Python task runner.

```
ProcessBuilder processBuilder = new ProcessBuilder(
    pythonExecutable(),
    taskRunnerPath(),
    taskType,
    inputDataJson
);

Process process = processBuilder.start();
boolean finished = process.waitFor(TIMEOUT.toMillis(), TimeUnit.MILLISECONDS);
if (!finished) {
    process.destroyForcibly();
    throw new TaskExecutionException("Python task timed out: " + taskType);
}
```

Listing 5: Starting a Python-backed task from Java.

Python-backed tasks are executed as a separate process. Listing 5 shows the bridge used by the Java executor. The task type and JSON input are passed as command-line arguments, and a timeout prevents the Python process from running indefinitely.

7.5. Web Dashboard

The web dashboard provides a visual interface for demonstration and manual inspection. It does not replace the JADE agents. Instead, it reads from and writes to the same PostgreSQL task schema. The dashboard is implemented in web/app.py using FastAPI routes, SQLAlchemy database sessions, Jinja templates, and static CSS files.

The main dashboard page shows task counts by status, active executor workloads, recent lifecycle events, and recent tasks. The task queue page supports status filtering and text search. The task detail page shows the selected task's execution state, input data, result, error message, timestamps, and lifecycle event timeline. The new-task page allows a user to create supported task types manually by entering task type, priority, maximum attempts, and JSON input data.

The dashboard is useful for demonstrations because it makes the internal state of the multi-agent system visible. A user can create tasks through the interface, start the JADE agents separately, and then observe how tasks move through PENDING, ASSIGNED, RUNNING, and terminal states. This separation is important: the dashboard provides visibility and task creation, while the coordinator and executor agents remain responsible for scheduling and execution.

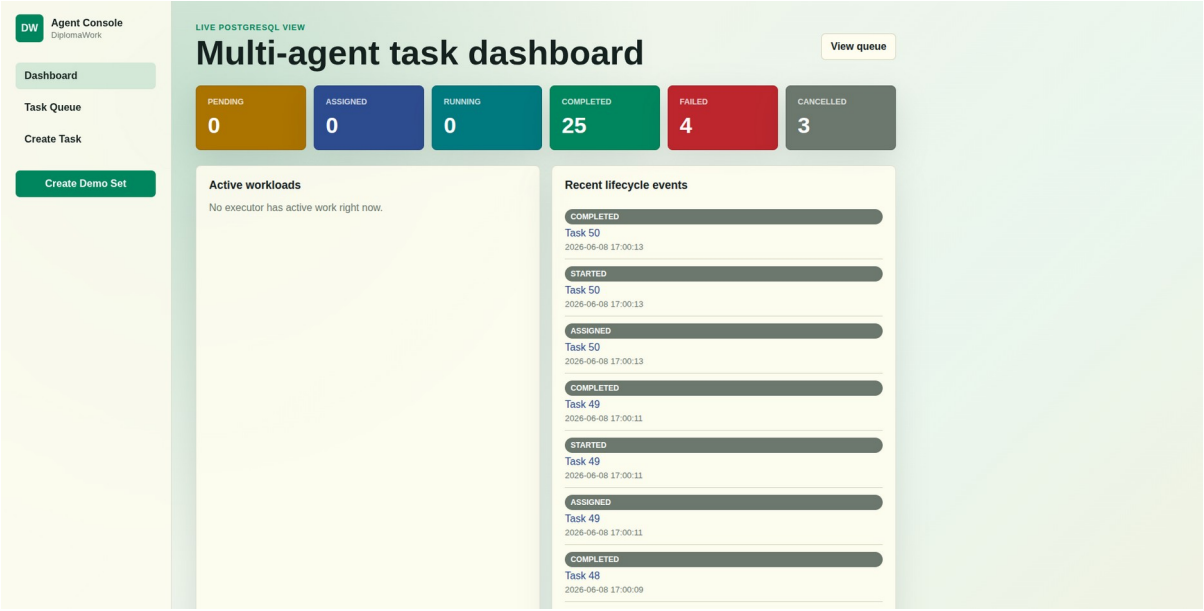

Figure 4: Web dashboard overview.

7.6. Running and Testing the System

The implementation includes scripts for common development and demonstration tasks. The run_agents.sh script starts two executor agents and one coordinator agent in a JADE platform. The run_web.sh script starts the FastAPI dashboard. The create_task.py and demo_create_tasks.sh scripts insert tasks into PostgreSQL, while monitoring scripts list tasks and lifecycle events.

The implementation also includes automated tests. Java unit tests verify task execution behaviour such as ECHO, TEXT_SUMMARY, invalid input handling, and unsupported task types. Python tests verify the Python-backed task modules, including text statistics and keyword counting. The repository also contains opt-in database integration tests for task lifecycle transitions and event recording. In the current local check, the Java test suite completed successfully with 7 active tests and 1 skipped opt-in database integration test, and the Python task module test suite completed successfully with 7 tests.

8. DEMONSTRATION AND RESULTS

This chapter demonstrates the implemented multi-agent task scheduling system and presents the observed results. The goal of the demonstration is to show that tasks can be

created, stored in PostgreSQL, assigned by the coordinator agent, executed by executor agents, and inspected through the web dashboard. The demonstration focuses on functional behaviour rather than performance benchmarking.

8.1. Demonstration Setup

The demonstration uses one PostgreSQL database, one coordinator agent, two executor agents, and the FastAPI web dashboard. The PostgreSQL database stores the task queue and lifecycle history. The coordinator and executor agents are started through the run_agents.sh script. The web dashboard is started separately through the run_web.sh script and reads the same database state.

Before running the demonstration, the database schema is prepared with the migration script. This creates or updates the tasks table, the task_events table, and the trigger used for lifecycle event recording. The demonstration can then be observed either through command-line monitoring scripts or through the web dashboard.

The demonstration setup can be summarized as follows:

1. Start PostgreSQL and apply the database migration.
2. Start the web dashboard.
3. Create demonstration tasks.
4. Start the JADE agents.
5. Observe task status changes and lifecycle events.
6. Inspect final task results.

8.2. Demonstration Task Set

The demonstration uses five task types supported by the system. These tasks cover both Java-native execution and Python-backed execution. The Java-native task types are ECHO and TEXT_SUMMARY. The Python-backed task types are PYTHON_ECHO, TEXT_STATS, and KEYWORD_COUNT.

Task type	Runtime	Purpose
ECHO	Java	Returns the message field from the input data
TEXT_SUMMARY	Java	Normalizes and truncates input text
PYTHON_ECHO	Python	Returns the message field through Python task runner
TEXT_STATS	Python	Counts characters, words, and lines
KEYWORD_COUNT	Python	Counts whole-word keyword matches in text

Table 5: Demonstration task types and expected results.

The tasks are inserted into the database with different priorities. This makes it possible to observe that the coordinator reads pending tasks according to priority ordering. Higher-priority tasks are selected before lower-priority tasks when they are available for scheduling.

8.3. Execution Results

After the demonstration tasks are created, they initially appear with the PENDING status. When the coordinator agent starts, it reads pending tasks from PostgreSQL and assigns them to the configured executor agents. The selected executor receives a JADE ACL REQUEST message containing the task id. The executor then verifies the task assignment in the database, marks the task as RUNNING, executes the task logic, and stores the final result.

In a successful demonstration run, the normal task flow is:

PENDING -> ASSIGNED -> RUNNING -> COMPLETED

The final results show that both Java-native and Python-backed task execution paths work correctly. Java tasks are executed directly by the TaskExecutor class, while Python-backed tasks are executed through the PythonTaskRunner bridge. The Python bridge starts the Python task runner script as a separate process and returns its output to the Java executor.

The demonstrated results confirm that the system can:

1. store tasks persistently in PostgreSQL;

2. select pending tasks according to priority;

3. assign tasks to executor agents;

4. communicate task assignments through JADE ACL messages;

5. execute Java and Python task logic;

6. store task results;

7. record lifecycle events for later inspection.

8.4. Dashboard Observation

The web dashboard provides a visual way to inspect the demonstration. The dashboard shows task counts grouped by status, active executor workloads, recent lifecycle events, and recent tasks. This makes it possible to observe the system without directly querying the database.

During the demonstration, newly created tasks first appear as PENDING. After the agents are started, the dashboard shows the tasks moving through ASSIGNED and RUNNING states. Completed tasks then appear with the COMPLETED status. The task detail page provides a more detailed view of an individual task, including its input data, result, assigned executor, timestamps, and lifecycle event timeline.

The lifecycle event timeline is especially useful because it confirms that status transitions are recorded separately from the task row itself. A successfully completed task contains events such as CREATED, ASSIGNED, STARTED, and COMPLETED. This shows that the system not only changes task status, but also records a traceable history of execution.

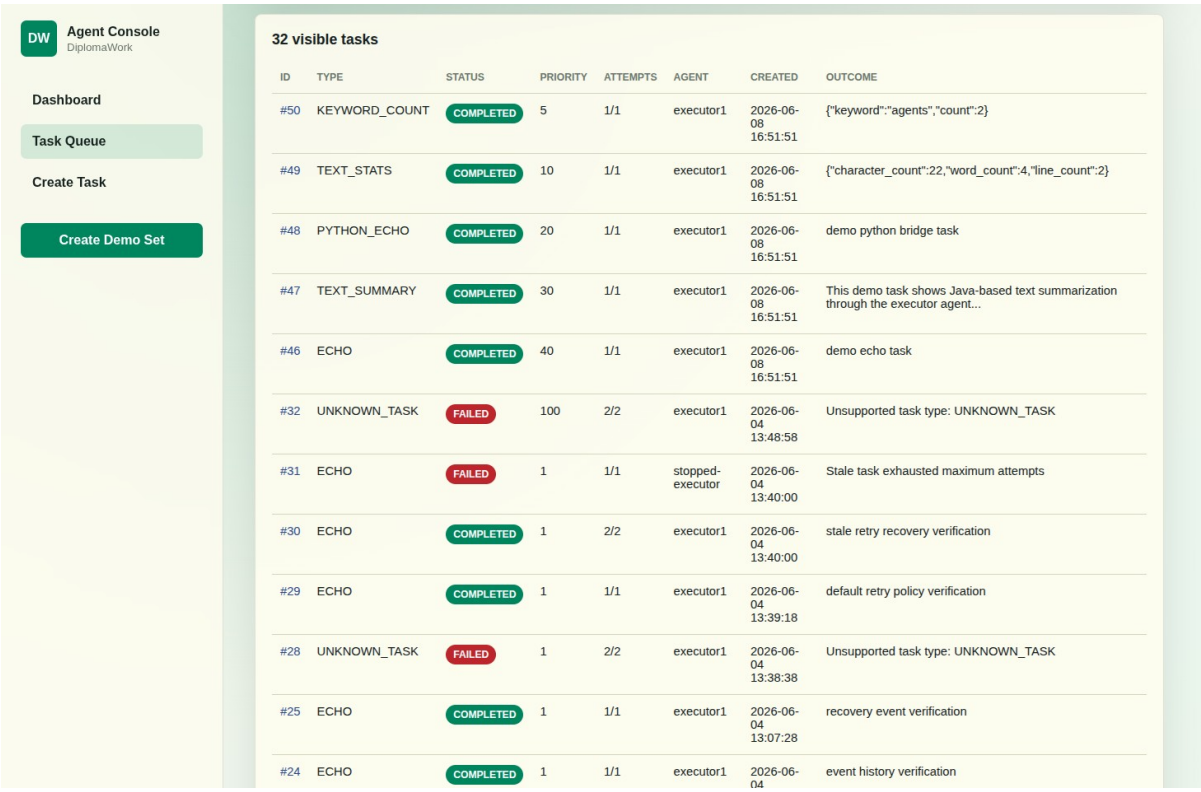

Figure 5: Task queue view showing completed demonstration tasks.

8.5. Test Results

The implementation was also checked through automated tests. The Java test suite verifies task execution behaviour for Java-native task types, including successful execution, invalid input handling, text summarization, malformed JSON, and unsupported task types. The Python test suite verifies Python-backed task logic, including echo behaviour, text statistics, keyword counting, empty input handling, and unsupported Python task types.

In the current local test run, the Java test suite completed successfully with seven active tests. One database integration test class was present but skipped because the opt-in database integration environment variable was not enabled. The Python task module test suite also completed successfully with seven tests.

Test group	Result
Java task executor	7 active tests passed
Database integration	1 opt-in test skipped
Python task modules	7 tests passed

Table 6: Automated test results.

These results support the functional demonstration by showing that the core execution logic behaves correctly in isolated tests. The skipped database integration test does not indicate a failed test; it is intentionally disabled unless the database integration test environment is explicitly enabled.

9. DISCUSSION

This chapter discusses the strengths and limitations of the implemented system. The demonstration in Chapter 8 shows that the system satisfies its main functional goal: tasks can be stored persistently, scheduled by a coordinator agent, assigned through JADE ACL messages, executed by executor agents, and inspected through both database records and the web dashboard. At the same time, the implementation intentionally remains limited in scope, because the goal of the thesis is to demonstrate a clear multi-agent scheduling workflow rather than build a production-grade distributed scheduler.

9.1. Strengths of the Implemented System

One strength of the system is the separation of scheduling and execution responsibilities. The coordinator agent does not execute task logic directly. Instead, it observes the queue, selects tasks, chooses an executor, and sends an assignment message. Executor agents are responsible for starting tasks, running task logic, and recording results. This separation makes the agent roles clear and supports the main multi-agent concept demonstrated by the project.

Another strength is the use of PostgreSQL as the persistent source of truth. Task status, assignment, attempts, results, errors, and timestamps are stored in the database. This makes the system easier to inspect and recover after interruptions. The use of conditional

database updates also protects important lifecycle transitions. For example, an executor can only start a task if the database confirms that the task is assigned to that executor.

The system also supports traceability through lifecycle events. The task_events table records important status changes such as creation, assignment, execution start, completion, retry scheduling, recovery, failure, and cancellation. This makes the system more transparent because the final task status is not the only available information. A user can inspect how the task reached that status.

A further strength is that the execution layer supports both Java-native and Python-backed task types. This shows that the executor agent can coordinate different execution mechanisms while keeping a common task lifecycle. The Python bridge is simple, but it demonstrates how the system can call task logic implemented outside the Java agent layer.

The web dashboard improves demonstrability. It allows a user to observe task counts, executor workload, lifecycle events, task results, and task details without manually querying PostgreSQL. This is useful for explaining and presenting the system because the behaviour of the agents becomes visible while the agents are running.

9.2. Limitations

The current scheduling strategy is intentionally simple. The coordinator selects the first pending task according to priority and creation time, then assigns it to the executor with the lowest number of active tasks. This demonstrates load-aware scheduling, but it does not estimate task duration, task complexity, executor speed, deadlines, or resource requirements. Therefore, it may not produce optimal results for heterogeneous or long-running workloads.

The system also uses a centralized coordinator. This makes the design easier to understand and demonstrate, but it creates a single scheduling point. If the coordinator is not running, new pending tasks will not be assigned. Executor agents can complete tasks already assigned to them, but the system depends on the coordinator for scheduling new work and recovering stale active tasks.

Another limitation is that executor agents are configured manually. The current implementation does not use the JADE Directory Facilitator for dynamic discovery of available executors. As a result, the coordinator receives executor names through startup arguments. This is acceptable for the local demonstration, but a larger multi-agent system would benefit from dynamic registration and discovery.

The implementation focuses on independent tasks. It does not support task dependencies, workflows, or tasks that require the output of previous tasks. This keeps the lifecycle clear, but it limits the types of workloads that can be represented. More complex scheduling problems would require dependency tracking and additional lifecycle rules.

The system is also demonstrated in a local environment. It does not currently evaluate behaviour across multiple physical machines or distributed JADE containers. The current implementation is therefore better understood as a local multi-agent prototype rather than a fully distributed deployment.

Finally, the failure handling mechanism is functional but limited. Failed tasks are retried according to a maximum attempt count, and stale active tasks can be recovered. However, the system does not classify failure causes, apply exponential backoff, notify users, or choose a different executor based on previous failures.

9.3. Possible Future Improvements

A natural future improvement is dynamic executor discovery through the JADE Directory Facilitator. Executor agents could register their capabilities, and the coordinator could discover available executors at runtime instead of receiving fixed names at startup. This would make the system more flexible and closer to a dynamic multi-agent architecture.

Another improvement would be capability-aware scheduling. At the moment, all configured executors are treated as equivalent. In a more advanced version, executors could advertise which task types they support, their current capacity, or their preferred runtime. The coordinator could then assign tasks based not only on load, but also on executor capability.

The scheduling strategy could also be extended. Possible extensions include deadline-aware scheduling, task-duration estimation, priority aging, weighted executor selection, and historical performance tracking. These strategies would make the scheduler more suitable for realistic workloads where tasks differ in cost and urgency.

The task model could be expanded to support dependencies between tasks. This would allow the system to represent workflows in which one task can start only after another task has completed. Such a feature would require additional dependency tables, dependency-aware scheduling, and new lifecycle rules for blocked or waiting tasks.

The monitoring layer could also be improved. The current web dashboard is useful for demonstration, but it could be extended with real-time updates, charts, executor status indicators, retry history, and clearer filtering of lifecycle events. This would make the system easier to operate and analyze.

Finally, the deployment model could be expanded beyond the local demonstration. Running JADE containers and executors on different machines would allow the system to demonstrate a more distributed architecture. This would require additional configuration, network setup, and testing, but it would better show how multi-agent systems can coordinate work across separate execution environments.

10. CONCLUSION

This thesis presented the design and implementation of a multi-agent task scheduling and execution system. The main goal was to demonstrate how agent-oriented concepts can be applied to a practical task processing problem. The implemented system separates scheduling and execution into two agent roles: a coordinator agent and executor agents.

The coordinator agent is responsible for selecting pending tasks, choosing an executor according to active workload, assigning the task in PostgreSQL, and sending a JADE ACL message. Executor agents receive assignment messages, verify task ownership through the database, execute the selected task logic, and record the final result or failure state.

The system uses PostgreSQL as the persistent source of truth for task data and lifecycle state. This allows tasks to remain inspectable after execution and recoverable after interruptions. The task lifecycle includes pending, assigned, running, completed, failed, and cancelled states. Retry handling and stale-task recovery make the workflow more robust within the local demonstration scope.

The implementation also shows that executor agents can coordinate different execution mechanisms. Some task types are executed directly in Java, while others are delegated to Python through a process-based bridge. This demonstrates that the agent layer can manage task lifecycle and coordination while allowing task-specific logic to be implemented in another runtime.

A FastAPI web dashboard was implemented to make the system observable. It allows tasks to be created and inspected, shows task status counts, displays executor

workload, and provides access to lifecycle events. This improves the demonstrability of the system because the behaviour of the agents can be observed without manually querying the database.

The final system satisfies the main goals defined in the thesis. It supports priority-based task selection, load-aware executor assignment, JADE-based message exchange, Java and Python task execution, persistent lifecycle state, retry handling, stale-task recovery, automated tests, and dashboard-based inspection.

The project remains intentionally limited in scope. It does not implement task dependencies, dynamic executor discovery through the JADE Directory Facilitator, distributed deployment across multiple machines, deadline-aware scheduling, or capability-aware executor selection. These limitations keep the implementation understandable while also identifying clear directions for future work.

Future development could extend the scheduler with dynamic executor discovery, executor capability registration, task dependency support, historical performance tracking, and a more advanced monitoring dashboard. The system could also be deployed across multiple JADE containers to demonstrate a more distributed multi-agent environment.

In conclusion, the thesis demonstrates that a multi-agent architecture can be used to build a clear and traceable task scheduling system. By combining JADE agents, PostgreSQL persistence, Java and Python execution logic, and a web dashboard, the project provides a complete prototype for studying task coordination, message exchange, and lifecycle management in a multi-agent setting.

SOURCES

[1] M. Wooldridge and N. R. Jennings, “Intelligent Agents: Theory and Practice,” The Knowledge Engineering Review, vol. 10, no. 2, pp. 115–152, 1995. DOI: 10.1017/S0269888900008122

[2] F. Zambonelli, N. R. Jennings, and M. Wooldridge, “Developing Multiagent Systems: The Gaia Methodology,” ACM Transactions on Software Engineering and Methodology, vol. 12, no. 3, pp. 317–370, 2003. DOI: 10.1145/958961.958963

[3] A. Y. Zomaya and Y.-H. Teh, “Observations on Using Genetic Algorithms for Dynamic Load-Balancing,” IEEE Transactions on Parallel and Distributed Systems, vol. 12, no. 9, pp. 899–911, 2001. DOI: 10.1109/71.954632

[4] PostgreSQL Global Development Group, “Transactions,” PostgreSQL 16 Documentation. [Online]. Available: https://www.postgresql.org/docs/16/tutorial-transactions.html. Accessed: June 5, 2026.

[5] F. Bellifemine, G. Caire, A. Poggi, and G. Rimassa, “JADE: A Software Framework for Developing Multi-Agent Applications—Lessons Learned,” Information and Software Technology, vol. 50, no. 1–2, pp. 10–21, 2008. DOI: 10.1016/j.infsof.2007.10.008

[6] L. F. Bittencourt, A. Goldman, E. R. M. Madeira, N. L. S. da Fonseca, and R. Sakellariou, “Scheduling in distributed systems: A cloud computing perspective,” Computer Science Review, vol. 30, pp. 31–54, 2018. DOI: 10.1016/j.cosrev.2018.08.002

[7] T. L. Casavant and J. G. Kuhl, “A taxonomy of scheduling in general-purpose distributed computing systems,” IEEE Transactions on Software Engineering, vol. 14, no. 2, pp. 141–154, 1988. DOI: 10.1109/32.4634

[8] O. Beaumont, L. Carter, J. Ferrante, A. Legrand, L. Marchal, and Y. Robert, “Centralized versus Distributed Schedulers for Bag-of-Tasks Applications,” IEEE Transactions on Parallel and Distributed Systems, vol. 19, no. 5, pp. 698–709, 2008. DOI: 10.1109/TPDS.2007.70747

[9] Telecom Italia / JADE, “Package jade.lang.acl,” JADE API Documentation. [Online]. Available: https://jade.tilab.com/doc/api/jade/lang/acl/package-summary.html. Accessed: June 5, 2026.

[10] PostgreSQL Global Development Group, “Overview of Trigger Behaviour,” PostgreSQL 16 Documentation. [Online]. Available: https://www.postgresql.org/docs/16/trigger-definition.html. Accessed: June 5, 2026.

[11] PostgreSQL JDBC Team, “Documentation,” PostgreSQL JDBC Driver. [Online]. Available: https://jdbc.postgresql.org/documentation/. Accessed: June 11, 2026.

[12] SQLAlchemy, “Session Basics,” SQLAlchemy 2.0 Documentation. [Online]. Available: https://docs.sqlalchemy.org/en/20/orm/session_basics.html. Accessed: June 11, 2026.

[13] S. Ramírez, “FastAPI,” FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/. Accessed: June 11, 2026.

[14] S. Ramírez, “Templates,” FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/advanced/templates/. Accessed: June 11, 2026.

[15] Celery Project, “Introduction to Celery,” Celery Documentation. [Online]. Available: https://docs.celeryq.dev/en/stable/getting-started/introduction.html. Accessed: June 11, 2026.

[16] PostgreSQL Global Development Group, “Explicit Locking,” PostgreSQL 16 Documentation. [Online]. Available: https://www.postgresql.org/docs/16/explicit-locking.html. Accessed: June 11, 2026.

[17] Oracle, “Class ProcessBuilder,” Java SE 17 & JDK 17 API Documentation. [Online]. Available: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ProcessBuilder.html. Accessed: June 11, 2026.

[18] Apache Software Foundation, “Maven Documentation,” Apache Maven. [Online]. Available: https://maven.apache.org/guides/index.html. Accessed: June 11, 2026.

[19] Pallets, “Template Designer Documentation,” Jinja Documentation. [Online]. Available: https://jinja.palletsprojects.com/en/stable/templates/. Accessed: June 11, 2026.

APPENDIX

1. Source code - https://github.com/AnsorTag/DiplomaProject.git